
PlacePrep

Data Acquisition & Storage Pipeline
Complete Technical Documentation

Data Sources · Legal Framework · Collection Strategy
Deduplication · ETL Pipeline · Storage Architecture
Implementation Checklist

Version 1.0 | June 2026
Newton School of Technology, Rishihood University
Project: PlacePrep — NST Interview Intelligence Portal

CLASSIFICATION: CONFIDENTIAL — Internal Use Only

Table of Contents

- 1. Document Overview & Purpose**
- 2. Legal & Compliance Framework**
 - 2.1 Group A — GitHub & Dataset Sources
 - 2.2 Group B — Static Web Scraping
 - 2.3 Group C — Dynamic Scraping & APIs
 - 2.4 Group D — Manual Collection
 - 2.5 Commercial Launch Policy
- 3. Section 1: All 52 Data Sources**
 - 3.1 Group A — Direct Download (Sources 1–32)
 - 3.2 Group B — Static Scraping (Sources 33–42)
 - 3.3 Group C — Dynamic Scraping (Sources 43–49)
 - 3.4 Group D — Manual Collection (Sources 50–52)
- 4. Section 2: Data Collection Strategy**
 - 4.1 Implementation Order & Phasing
 - 4.2 Group A — Direct Download Procedures
 - 4.3 Group B — BeautifulSoup Implementation
 - 4.4 Group C — Reddit API & Playwright
 - 4.5 Group D — Manual Pipeline
- 5. Deduplication & Conflict Resolution**
 - 5.1 Identified Overlapping Source Pairs
 - 5.2 Source Priority Matrix
 - 5.3 Deduplication Logic with pg_trgm
- 6. Section 3: ETL Pipeline (Apache Airflow)**
 - 6.1 Pipeline Architecture & Airflow DAG
 - 6.2 The 8-Step Processing Flow
- 7. Section 4: Storage Architecture**
 - 7.1 PostgreSQL — Primary Database & Schema
 - 7.2 Elasticsearch — Search Engine
 - 7.3 pgvector — AI / Vector Database
 - 7.4 Redis — Cache Layer
- 8. Implementation Checklist & Execution Order**

1. Document Overview & Purpose

This document is the complete technical reference for the data acquisition and storage pipeline of PlacePrep — NST's Interview Intelligence Portal. It is intended for the developer assigned to build the data scraping, ETL, and storage layer of the platform.

This document must be read in full before writing a single line of scraping code. Every decision — which tool to use, which order to run sources, how to handle duplicates, and what is legally safe — is documented here.

1.1 Pipeline Stages Covered

Stage	Description	Ref.
Data Sources	52 curated origins of all interview data — GitHub repos, Kaggle datasets, scraping targets, APIs, and NST-internal sources.	Sec. 3
Data Collection	Tools and procedures for acquiring data: direct download, BeautifulSoup, Playwright, Reddit/Codeforces API, manual forms.	Sec. 4
ETL Pipeline	The 8-step Apache Airflow pipeline that transforms raw data into clean, classified, structured records ready for storage.	Sec. 6
Storage Architecture	The four storage systems — PostgreSQL, Elasticsearch, pgvector, and Redis — and what data lives in each.	Sec. 7

1.2 Key Conventions Used in This Document

- **[SAFE]** — No legal risk for current NST-internal use.
- **[CAUTION]** — Grey area; acceptable internally; must be reviewed before commercial launch.
- **[HIGH RISK]** — Do not use for public-facing or commercial use. Manual seed data only.
- **Priority 1** = Highest priority; implement first. Priority 4+ = Nice-to-have.
- All rate limits stated are **MAXIMUM** values. Always add randomised delay (never constant).

1.3 What This Document Does Not Cover

This document does not cover the FastAPI backend layer, the Next.js frontend, or the AI/RAG service. It ends at the point where clean, structured data is in storage and ready to be served by the API. Those layers are documented separately.

2. Legal & Compliance Framework

This is the most critical section of this document. Every developer working on data collection must read and follow this section. Failure to comply — especially during the public launch phase — could expose NST and PlacePrep to legal liability.

2.1 Universal Rules — Apply to All Sources

Non-Negotiable Scraping Rules

- Always check `/robots.txt` before scraping any website. If a path is Disallowed, do not scrape it.
- Set a descriptive User-Agent: "PlacePrep-Bot/1.0 (+https://nst.edu/placeprep)" — never impersonate a real browser.
- Rate limit: Never exceed 1 request per 3 seconds. Always use randomised delays (e.g., `random.uniform(3, 6)`).
- Never bypass login walls, CAPTCHAs, or authentication. This violates the Indian IT Act §43 and CFAA.
- Store only what you need. Delete raw HTML/JSON from the staging lake 30 days after ETL completes.
- Always populate the `source_url` field on every record — attribution is both ethical and legally protective.

2.2 Group A — GitHub Repos & Datasets (Sources 1–32)

[SAFE] GitHub, Kaggle, HuggingFace — Fully Safe for All Use

- All 26 GitHub repos are under open-source licenses (MIT, Apache 2.0, CC-BY). Educational use is explicitly permitted.
- Kaggle datasets (Sources 27–30) and HuggingFace datasets (31–32) are published for research/ML use. Downloading via their official APIs is fully legal and encouraged.
- Action: Before processing each repo, check its LICENSE file. Document the license in your source registry.
- CAVEAT for commercial launch: The underlying LeetCode/GFG question statements may carry separate copyright. For PlacePrep, store metadata and processed insights only (title, topic, difficulty, company tag) — rephrase question bodies rather than copying verbatim.

2.3 Group B — Static Web Scraping (Sources 33–42)

[CAUTION] GFG, IndiaBix, PrepInsta, CareerRide — Internal Use Only

- **GeeksForGeeks (33–34):** ToS §5 prohibits automated access. Indian Copyright Act §52 provides research/educational fair use. Acceptable for NST-internal use. Do NOT redistribute scraped GFG content.
- **IndiaBix (35–37):** Less restrictive ToS. Static HTML, no login required. Educational scraping is low risk. Always verify robots.txt crawl-delay directive.
- **PrepInsta (38–41):** ToS explicitly prohibits automated data collection. Use for NST-internal research only. MUST STOP scraping PrepInsta before any public/commercial launch.
- **CareerRide (42):** No explicit prohibition found. Static HTML pages. Low risk.
- Rate limit for all Group B: Max 1 request per 4 seconds, randomised. No login bypass under any circumstance.

2.4 Group C — Dynamic Scraping & APIs (Sources 43–49)

[SAFE] Reddit API & Codeforces API — Always Use the Official API

- **Reddit API (Sources 45–46):** Fully legal. Register a free developer app at reddit.com/prefs/apps. Free tier: 60 requests/min (OAuth). This is the correct approach — never Playwright-scrape Reddit when the API exists.
- **Codeforces API (Source 47):** Fully public and documented. No authentication required. Rate limit: 5 req/sec.
- These two sources must always be implemented FIRST — zero legal risk, structured data, no scraping required.

[CAUTION] Unstop (Source 49) — Public Listings Only

- Unstop is a startup-friendly platform. ToS discourages scraping but enforcement is light.
- Scope: Only scrape publicly visible competition listings. Do not attempt authenticated content.
- Check robots.txt at unstop.com/robots.txt before starting. Use 3–5s delays and realistic user-agent.

[HIGH RISK] AmbitionBox, InterviewBit, Naukri — Do Not Automate

- **AmbitionBox (Source 43):** Owned by InfoEdge/Naukri. ToS explicitly prohibits automated data collection. Use manual seed data only — tag records as `manually_curated=TRUE`. Build no automated pipeline.
- **InterviewBit (Source 44):** Requires account login. Scraping authenticated sessions is a direct ToS violation and potentially illegal under IT Act §43. Do not automate. Manually collect a representative seed sample instead.
- **Naukri.com (Source 48):** InfoEdge has historically taken legal action against scrapers. DO NOT scrape Naukri for any purpose. ALTERNATIVE: Replace with NST Placement Cell data (Source 51) which is safer and more relevant.

2.5 Group D — Manual Collection (Sources 50–52)

[SAFE] NST Forms, Placement Cell, Engineering Blogs — Full Ownership

- **NST Google Form (Source 50):** Data voluntarily submitted by NST students. Full ownership by NST/PlacePrep team.
- **NST Placement Cell Records (Source 51):** Institutional data. Obtain formal written permission from the Placement Cell before accessing. Restrict access to PlacePrep admins only.
- **Engineering Blogs (Source 52):** Public blog content used as research reference — not automated scraping. Reading and manually paraphrasing/summarising is fair use.

2.6 Commercial Launch Policy

When PlacePrep transitions from NST-internal to a publicly available platform, the following changes are mandatory before launch:

Pre-Launch Legal Checklist

- **STOP:** All Group B scraping (GFG, IndiaBix, PrepInsta, CareerRide) unless signed data licensing agreements exist.
- **STOP:** AmbitionBox and InterviewBit automated collection. Remove all such records or obtain explicit licensing.
- **STOP:** Naukri scraping entirely. Replace with licensed data or NST Placement Cell partnership.
- **CONTINUE:** Group A (verify individual repo licenses — MIT/Apache are fine), Group D (own data), Reddit API, Codeforces API.
- **INITIATE:** Reach out to GFG and AmbitionBox for formal data licensing. Both companies have precedent for B2B data partnerships (typically revenue share or attribution agreements).
- **ADD:** Legal disclosure in PlacePrep Terms of Use acknowledging all data sources and their respective licenses.

3. Section 1: All 52 Data Sources

The following tables list all 52 curated data sources organised by collection group. Legal status reflects NST-internal use. Priority 1 = implement first.

3.1 Group A — Direct Download (Sources 1–32)

No scraping required. Data obtained via git clone, CSV download, or Python API calls. Implement these first — zero legal risk, maximum data volume.

#	Source / Repository	Round Type	Access Method	Legal	Priority
1	LeetCode Company-wise Questions (May 2026) github.com/snehasishroy/...	DSA/Coding	git clone / ZIP	SAFE	1
2	LeetCode 30/60/90 Day Trends github.com/liquidslr/...	DSA/Coding	git clone	SAFE	2
3	LeetCode Frequency Sorted (19.8k stars) github.com/krishnadey30/...	DSA/Coding	git clone	SAFE	2
4	LeetCode Company-wise PDF github.com/Faizi-alpha/...	DSA/Coding	PDF Download	SAFE	4
5	FAANG Coding Interview Questions github.com/ombharatiya/...	DSA/Coding	git clone	SAFE	2
6	DSA Must-Do + Editorials github.com/ankushchimnani/...	DSA/Coding	git clone	SAFE	3
7	Coding Interview University (200k+ stars) github.com/jwasham/...	DSA Study Plan	Copy from README	SAFE	2
8	Interviews Repo + Solutions github.com/kdn251/...	DSA/Coding	git clone	SAFE	3
9	System Design Primer (200k+ stars) github.com/donnemartin/...	System Design	Copy from README	SAFE	1
10	Awesome System Design Resources github.com/ashishps1/...	System Design	Copy from README	SAFE	1
11	Low Level Design Primer github.com/prasadgujar/...	LLD Round	Copy from README	SAFE	2
12	ReactJS Interview Questions 500+ github.com/sudheerj/reactjs-...	Frontend Tech	git clone	SAFE	2
13	JavaScript Interview Questions 500+ github.com/sudheerj/javascript-...	Frontend Tech	git clone	SAFE	2
14	Frontend Interview Handbook github.com/yangshun/...	Frontend Tech	Copy from README	SAFE	2
15	NodeJS Interview Questions (188 Q) github.com/ajay-jagtap/...	Backend Tech	git clone	SAFE	2
16	Backend NodeJS + MongoDB + Golang QA github.com/Gauthamim007/...	Backend Tech	git clone	SAFE	3

#	Source / Repository	Round Type	Access Method	Legal	Priority
17	Core CS — OS + Networks + DBMS github.com/workattech/...	Technical Round	Copy from README	SAFE	1
18	SQL + DBMS Interview Questions github.com/xoraus/...	Technical Round	Copy from README	SAFE	1
19	All CS Subjects One Repo github.com/avinash201199/...	Technical Round	Copy from README	SAFE	2
20	Awesome Behavioral Interviews (40+ STAR Qs) github.com/ashishps1/...	HR Round	Copy from README	SAFE	2
21	Behavioral Question List github.com/gregorojstersek/...	HR Round	Copy from README	SAFE	2
22	HR Interview Resources github.com/Lakhankumawat/...	HR Round	Copy from README	SAFE	3
23	CSE Aptitude 6,000+ Questions github.com/rohanmistry231/...	Aptitude Round	git clone	SAFE	1
24	TCS + Infosys Placement Handbook github.com/rubanthilak/...	All Rounds	git clone	SAFE	1
25	TCS + AMCAT Coding Questions github.com/packetprep/...	Coding Round	git clone	SAFE	1
26	ML Interview Questions github.com/andrewekhale/...	ML Technical	git clone	SAFE	2
27	LeetCode Problem Dataset (Kaggle) kaggle.com/datasets/gzipchrist/...	DSA/Coding	CSV Download	SAFE	1
28	LeetCode Easy Dataset (Kaggle) kaggle.com/datasets/akshatsharma0610/...	DSA/Coding	CSV Download	SAFE	3
29	LeetCode Medium Dataset (Kaggle) kaggle.com/datasets/akshatsharma0610/...	DSA/Coding	CSV Download	SAFE	3
30	Data Science Interview Dataset (Kaggle) kaggle.com/datasets/die9origephit/...	DS/ML Technical	CSV Download	SAFE	2
31	Interview Qs with Context (HuggingFace) huggingface.co/datasets/andmev/...	General Tech	load_dataset()	SAFE	3
32	AI Interview Questions (HuggingFace) huggingface.co/datasets/K-areem/...	AI/ML Technical	load_dataset()	SAFE	2

Note: Source 4 (PDF) is a low-priority backup for Sources 1–3. Sources 28–29 (Kaggle Easy/Medium) supplement Source 27 which covers the full dataset. See §5 for the complete deduplication strategy across these overlapping sources.

3.2 Group B — Static Scraping Sources (Sources 33–42)

Publicly accessible, static HTML pages. BeautifulSoup is sufficient. Implement after Group A. Always verify robots.txt before starting each source.

#	Source	Round Type	URL Pattern	Legal Status	Priority
33	GeeksForGeeks Interview Experiences geeksforgeeks.org/company-interview-corner/	All Rounds	/company-interview-corner/	CAUTION	1
34	GeeksForGeeks Company DSA Lists geeksforgeeks.org	Coding Round	/must-do-coding-questions/	CAUTION	2
35	IndiaBix Placement Papers indiabix.com/placement-papers/	Aptitude	/placement-papers/companies/	CAUTION	1
36	IndiaBix Aptitude Questions indiabix.com/aptitude/	Aptitude	/aptitude/questions-and-answers/	CAUTION	1
37	IndiaBix Technical MCQs indiabix.com/technical/	Technical	/technical/	CAUTION	2
38	PrepInsta — TCS prepinsta.com/tcs/	Aptitude+Coding	/tcs/	CAUTION	2
39	PrepInsta — Infosys prepinsta.com/infosys/	Aptitude+Coding	/infosys/	CAUTION	2
40	PrepInsta — Wipro prepinsta.com/wipro/	Aptitude+Coding	/wipro/	CAUTION	3
41	PrepInsta — All Companies prepinsta.com/placement-papers/	Aptitude	/placement-papers/	CAUTION	3
42	CareerRide careerride.com/interview-questions.aspx	Technical+HR	/interview-questions.aspx	CAUTION	3

3.3 Group C — Dynamic Scraping Sources (Sources 43–49)

JavaScript-rendered pages or API-based sources. Reddit and Codeforces have official APIs — always prefer the API over scraping when it exists.

#	Source	Round Type	Tool	Legal Status	Priority
43	AmbitionBox ambitionbox.com/interviews	All Rounds	Manual seed only	HIGH RISK	2
44	InterviewBit Company-wise interviewbit.com/companies/	All Rounds	Manual seed only	HIGH RISK	3
45	Reddit r/cscareerquestions	All Rounds	Reddit API (Free)	SAFE	1
46	Reddit r/developersIndia	All Rounds	Reddit API (Free)	SAFE	1
47	Codeforces API codeforces.com/api/problemset.problems	Competitive Prog.	Official API (Free)	SAFE	2
48	Naukri.com Jobs naukri.com/software-engineer-jobs	Skills Data	DO NOT SCRAPE	HIGH RISK	—
49	Unstop Competitions unstop.com/competitions	Coding Challenges	Playwright	CAUTION	3

3.4 Group D — Manual Collection Sources (Sources 50–52)

Not scraped. This is the most valuable data in PlacePrep — unique to NST and impossible to obtain from any external source.

#	Source	Round Type	Method	Legal	Key Notes
50	NST Seniors Google Form	All Rounds	Google Form (CSV export)	SAFE	HIGHEST PRIORITY. Unique NST experiences. Design and distribute to all students.
51	NST Placement Cell Records	All Rounds	Direct institutional request	SAFE	Historical company/role/package data. Requires written permission from placement office.
52	Company Engineering Blogs (Flipkart, Zomato, Razorpay, Zepto)	System Design	Manual reading + curation	SAFE	Hiring process insights. Manual notes only — no automated scraping.

4. Section 2: Data Collection Strategy

This section provides the step-by-step implementation guide for each collection group. Follow the implementation order: Group D → Group A → Group C APIs → Group B → Group C Playwright. Start with zero-risk, highest-value sources.

4.1 Implementation Order & Phasing

Recommended Phased Rollout

- **Phase 1 (Week 1):** Group D — Set up NST Google Form; request Placement Cell data. Zero risk, maximum uniqueness value.
- **Phase 2 (Week 1–2):** Group A — git clone all repos; download Kaggle CSVs; run HuggingFace load_dataset(). Bulk of structured data, zero legal risk.
- **Phase 3 (Week 2):** Group C APIs — Set up Reddit API (OAuth); call Codeforces API. Official APIs, clean structured JSON data.
- **Phase 4 (Week 3):** Group B — BeautifulSoup scrapers for IndiaBix, CareerRide, GFG, PrepInsta. Respect robots.txt and rate limits.
- **Phase 5 (Week 3–4):** Group C Playwright — AmbitionBox and Unstop only. Skip Naukri and InterviewBit entirely.

4.2 Group A — Direct Download Procedures

GitHub Repos (Sources 1–26)

Clone all repos to a local staging directory. The ETL pipeline will pick up all .md, .csv, .json, and .txt files. Never modify cloned files directly.

```
# Clone to staging directory with source ID as folder name
git clone https://github.com/snehasishroy/leetcode-companywise-interview-questions \
  /data/staging/github/01/
git clone https://github.com/donnemartin/system-design-primer \
  /data/staging/github/09/
# Repeat for all 26 repos using their respective source IDs
```

Kaggle Datasets (Sources 27–30)

Install the Kaggle CLI and configure your API token. Download directly to staging.

```
pip install kaggle --break-system-packages
# Place kaggle.json from kaggle.com/account in ~/.kaggle/
kaggle datasets download -d gzipchrist/leetcode-problem-dataset \
  -p /data/staging/kaggle/27/ --unzip
kaggle datasets download -d die9origephit/data-science-interview-questions \
  -p /data/staging/kaggle/30/ --unzip
```

HuggingFace Datasets (Sources 31–32)

```

from datasets import load_dataset # pip install datasets

ds31 = load_dataset("andmev/interview-question-with-context")
ds32 = load_dataset("K-areem/AI-Interview-Questions")

# Export to CSV for ETL pipeline
ds31["train"].to_csv("/data/staging/hf/31/data.csv", index=False)
ds32["train"].to_csv("/data/staging/hf/32/data.csv", index=False)

```

4.3 Group B — BeautifulSoup Implementation

All Group B scrapers share the same base pattern. Build a common BaseScraper class and extend it per source.

```

import requests, time, random, urllib.robotparser
from bs4 import BeautifulSoup

class BaseScraper:
    def __init__(self, source_id, base_url, delay=(3, 6)):
        self.source_id = source_id
        self.base_url = base_url
        self.delay = delay
        self.session = requests.Session()
        self.session.headers.update({
            "User-Agent": "PlacePrep-Bot/1.0 (+https://nst.edu/placeprep)",
        })

    def robots_allow(self, path):
        rp = urllib.robotparser.RobotFileParser()
        rp.set_url(f"{self.base_url}/robots.txt")
        rp.read()
        return rp.can_fetch("*", f"{self.base_url}{path}")

    def fetch(self, path):
        time.sleep(random.uniform(*self.delay)) # randomised delay
        resp = self.session.get(f"{self.base_url}{path}", timeout=15)
        resp.raise_for_status()
        return BeautifulSoup(resp.text, "html.parser")

    def run(self):
        raise NotImplementedError("Implement per-source scraper")

```

Per-Source Notes for Group B

- **GFG (33):** URL pattern — /company-interview-corner/page/{n}/. Extract company name, round description, questions. Skip login-gated pages.
- **IndiaBix Aptitude (36):** Each question page has clean structure — .question div, .options list, .answer div. Very reliable HTML; extract and store as-is.
- **PrepInsta (38–41):** Per-company pages list topics. Follow links to individual question pages. HTML structure varies per company page — inspect before parsing.
- **CareerRide (42):** List page at /interview-questions.aspx gives company + tech links. Each link leads to a clean Q&A page. Simple two-column layout.
- **All Group B:** Always call self.robots_allow(path) before scraping any new path. Log and skip disallowed paths.

4.4 Group C — Reddit API & Playwright Implementation

Reddit API Setup (Sources 45, 46) — Implement First

```
import praw # pip install praw
import os

reddit = praw.Reddit(
    client_id=os.environ["REDDIT_CLIENT_ID"],
    client_secret=os.environ["REDDIT_CLIENT_SECRET"],
    user_agent="PlacePrep-Bot/1.0 by NST"
)

# Collect top posts from last year
for post in reddit.subreddit("cscareerquestions").top(
    limit=1000, time_filter="year"):
    if "interview" in post.title.lower():
        # Extract: title, selftext, score, created_utc
        # Also pull top 10 comments per post
        for comment in post.comments[:10]:
            process_comment(comment.body)
```

Codeforces API (Source 47)

```
import requests

resp = requests.get(
    "https://codeforces.com/api/problemset.problems",
    params={"tags": "dp,graphs,implementation,greedy"},
    timeout=10
)
data = resp.json()
problems = data["result"]["problems"]
# Each problem: contestId, name, rating (difficulty), tags (topics)
# Map CF rating to PlacePrep difficulty: <1200=easy, 1200-1800=medium, >1800=hard
```

Playwright for AmbitionBox (Source 43) and Unstop (Source 49)

```

from playwright.async_api import async_playwright
import asyncio, random

async def scrape_company(company_slug: str):
    async with async_playwright() as pw:
        browser = await pw.chromium.launch(headless=True)
        ctx = await browser.new_context(
            user_agent="Mozilla/5.0 (Windows NT 10.0; Win64; x64) "
            "AppleWebKit/537.36 Chrome/120.0.0.0 Safari/537.36"
        )
        page = await ctx.new_page()
        url = f"https://www.ambitionbox.com/interviews/{company_slug}"
        await page.goto(url, wait_until="networkidle")
        await asyncio.sleep(random.uniform(3, 6)) # human-like delay

        # Extract visible interview cards only
        cards = await page.query_selector_all(".interview-experience-card")
        for card in cards:
            text = await card.inner_text()
            yield {"company": company_slug, "text": text, "source": 43}
        await browser.close()

```

4.5 Group D — Manual Pipeline

Google Form Design (Source 50) — Recommended Fields

- NST Batch Year (required), Branch (optional)
- Company Name (required), Role Applied For (required)
- Interview Date — approximate month/year
- Total Number of Rounds (number input)
- For each round: Round Type (dropdown: Aptitude / Coding / Technical / System Design / LLD / HR), Topics Covered (multi-select), Questions Asked (text area), Difficulty (1–5 stars), Outcome (Cleared / Not Cleared)
- Tips for Future Candidates (text area), Overall Experience Rating (1–5)
- Set form to accept NST email only — ensures data authenticity and quality.
- Export monthly as CSV → upload to s3://placeprep-staging/50/{month}/ → ETL runs automatically.

Placement Cell Request (Source 51)

- Send a formal written request (email + printed letter) to the NST Placement Cell.
- Request data: Company-wise placement history (2020–2025), roles offered, packages, rounds conducted, student selection count.
- Once received, structure as a standardised CSV (columns: year, company, role, package_lpa, rounds_count, students_selected, branch).
- Upload to s3://placeprep-staging/51/ and restrict DB access to admin role only.

5. Deduplication & Conflict Resolution

With 52 sources covering overlapping topics, duplicate data is inevitable. This section defines which sources overlap, which takes priority, and how the pipeline detects and merges duplicates without losing metadata.

5.1 Identified Overlapping Source Pairs

Overlapping Sources	Type of Overlap	Resolution
Sources 1–4 (GitHub) + Sources 27–29 (Kaggle)	All contain LeetCode company-wise questions. Same questions, different formats and metadata.	PRIMARY: Source 1 (most updated, June 2026) ENRICH WITH: Source 27 (adds acceptance rate + topic tags) TREND DATA: Source 2 (adds 30/60/90-day window) SKIP: Source 4 (PDF — same data as Source 1)
Sources 33–34 (GFG) vs. Sources 17–19 (GitHub CS)	GitHub repos have structured OS/DBMS/SQL notes. GFG has same topics as interview questions.	GitHub (17–19): Use for structured notes and question lists. GFG (33–34): Use for interview experiences ONLY. No cross-dedup needed — different content types.
Source 5 (FAANG Coding) vs. Sources 1–3 (LeetCode)	Source 5 is curated FAANG problems that already exist in Sources 1–3.	Sources 1–3 are primary. Source 5 adds context/tips. Run dedup check; merge context into existing records.
Source 23 (Aptitude 6,000+) vs. Sources 35–41 (IndiaBix, PrepInsta)	High content overlap — all contain TCS/Infosys/Wipro aptitude questions.	Source 23 is PRIMARY for aptitude (clean CSV format). Group B sources add company-specific attribution. Dedup by text similarity; keep company tags from both.
Sources 12–14 (Frontend GitHub) vs. Source 33 (GFG)	Frontend questions (React, JS, HTML) appear on both GFG and GitHub repos.	GitHub repos (12–14): PRIMARY for tech question bank. GFG (33): Adds real interview experience context. Dedup tech questions; treat experiences as a separate entity type.
Sources 20–22 (Behavioral GitHub) vs. Sources 45–46 (Reddit)	Classic HR questions appear in both GitHub banks and Reddit interview posts.	GitHub (20–22): Canonical HR question list — PRIMARY. Reddit: Real candidate experiences about HR rounds — supplementary. Both have unique value; minimal true deduplication needed.

5.2 Source Priority Matrix

When two sources contain the same question, the following priority order determines which record becomes the canonical (master) copy:

Priority	Source Type	Reason
1	NST Google Form + Placement Cell (Group D)	100% unique to NST. Highest relevance to NST students. Cannot be found anywhere else.
2	GitHub repos 50k+ stars (Sources 7, 9, 23)	Community-verified, actively maintained, high signal-to-noise ratio.

Priorit y	Source Type	Reason
3	Kaggle / HuggingFace datasets (Sources 27–32)	Machine-readable CSV format with rich metadata. Structured and clean.
4	Other GitHub repos (Sources 1–6, 8, 10–26)	Good quality but less curated. Use for supplementary data and metadata enrichment.
5	Reddit API (Sources 45–46)	Real student experiences. Unique content type. Not for question bank — for experience feed.
6	BeautifulSoup sources (Sources 33–42)	Lower reliability due to HTML parsing variance. Use only when no GitHub equivalent exists.
7	Playwright sources (Sources 43, 49)	Highest collection risk. Treat as supplementary only. Tag caution_source=TRUE in DB.

5.3 Deduplication Logic — pg_trgm

The deduplication step runs in ETL Step 4, after normalisation. PostgreSQL's pg_trgm extension provides fast fuzzy string matching on question text.

Step 1: Enable extension and create index

```
CREATE EXTENSION IF NOT EXISTS pg_trgm;

-- GiST index for fast trigram similarity lookups
CREATE INDEX idx_q_trgm ON interview_questions
  USING GiST (question_text gist_trgm_ops);

-- Scoped index for topic-level dedup (faster than full-table scan)
CREATE INDEX idx_q_topic ON interview_questions (topic_id);
```

Step 2: Find duplicate candidates within same topic

```
-- Find pairs with >85% text similarity in same topic
SELECT q1.id AS canonical_id,
       q2.id AS duplicate_id,
       q1.source_priority AS canonical_priority,
       q2.source_priority AS duplicate_priority,
       similarity(q1.question_text, q2.question_text) AS sim_score
FROM interview_questions q1
JOIN interview_questions q2
  ON q1.id < q2.id AND q1.topic_id = q2.topic_id
WHERE similarity(q1.question_text, q2.question_text) > 0.85;

-- Keep lower source_priority number (higher quality) as canonical
UPDATE interview_questions
  SET canonical_id = :canonical_id,
      is_duplicate = TRUE
WHERE id = :duplicate_id;
```

Step 3: Merge metadata from duplicate into canonical

```
-- Transfer company tags from duplicate to canonical record
INSERT INTO question_companies (question_id, company_id)
SELECT :canonical_id, company_id
FROM question_companies
WHERE question_id = :duplicate_id
ON CONFLICT DO NOTHING;

-- Merge frequency – always take the higher value
UPDATE interview_questions
  SET frequency_score = GREATEST(frequency_score, :dup_frequency)
WHERE id = :canonical_id;
```

Deduplication Configuration

- **Similarity threshold:** 0.85 (85% match). Catches near-duplicates without false positives. Tune down to 0.80 if too many missed duplicates are found after QA.
- **Scope:** Within same topic_id first (fast). Cross-topic dedup only for exact title match (>0.97 similarity).
- **After dedup:** canonical=TRUE records are used for all API queries. is_duplicate=TRUE records remain in DB for audit but are excluded from responses.
- **Schedule:** Dedup runs on new records after each ETL cycle. Full table re-scan runs every Sunday.
- **Exception:** Interview experiences are NEVER deduplicated. Each experience is unique by nature — even two experiences from the same company describe unique encounters.

6. Section 3: ETL Pipeline

The ETL (Extract, Transform, Load) pipeline is the cleaning factory between raw collected data and clean storage. All data — regardless of source — flows through this 8-step Apache Airflow pipeline before being stored.

6.1 Pipeline Architecture — Apache Airflow

Airflow DAG Configuration

DAG ID	placeprep_data_pipeline
Schedule	@daily for Group B/C scrapers; manually triggered for Group A (one-time bulk load)
Max Active Runs	1 — prevent parallel pipeline conflicts
Retry	2 attempts per task with 5-minute delay on failure
Alert	Email notification to PlacePrep admin on any DAG failure
Input path	s3://placeprep-staging/{source_id}/{YYYY-MM-DD}/
Output	Written directly to PostgreSQL (staging schema) → promoted to production after dedup
Log retention	30 days of task logs; raw files archived to s3://placeprep-archive/ after 30 days

6.2 The 8-Step Processing Flow

1 Step 1 — Raw Data Ingestion

All scraped/downloaded files (JSON, CSV, HTML, Markdown) are uploaded to the S3/MinIO raw data lake at s3://placeprep-staging/{source_id}/{date}/. Nothing is modified at this step — this is a pure dump. Airflow reads file paths from S3 and passes them to downstream tasks. Files are retained for 30 days then moved to s3://placeprep-archive/. Output: a list of S3 file paths for the next step.

2 Step 2 — Data Validation

Each file is checked for: (a) correct encoding (UTF-8), (b) minimum required fields not null (question_text or experience_text), (c) file size > 0 bytes, (d) valid JSON/CSV structure. Records failing validation are written to s3://placeprep-rejected/{date}/ with a reason_code field. The pipeline continues with valid records only. A daily validation summary is emailed to the admin showing pass/fail counts per source.

3 Step 3 — Data Cleaning

Raw web content is cleaned: strip all HTML tags using BeautifulSoup .get_text(), remove boilerplate text (ads, navbars, cookie banners, share buttons), fix encoding errors using the ftfy library, normalise Unicode to NFC form, collapse multiple whitespace into single spaces, remove null/empty strings. For GitHub Markdown content: strip Markdown syntax (headers #, bold **, code fences ```) and extract plain text. Output: clean plain text for every field.

4 Step 4 — Deduplication

Run `pg_trgm` similarity check (see §5.3) on new records against all existing PostgreSQL records. Threshold: 0.85 = duplicate. Scope: within same `topic_id` for questions; same company + high text similarity for experiences. Result: each new record is tagged `canonical=TRUE` or `is_duplicate=TRUE` with a reference to `canonical_id`. Only canonical records proceed to Elasticsearch indexing and pgvector embedding.

5 Step 5 — Normalisation

Standardise variable text fields to controlled vocabulary using lookup tables: (a) Company names — "Amazon Web Services" → "Amazon", "Meta Platforms" → "Meta". (b) Role names — "Software Dev Engineer" → "Software Development Engineer (SDE)". (c) Round types → controlled enum: [aptitude, coding, technical, system_design, lld, hr, managerial]. (d) Difficulty → [easy, medium, hard]. (e) Topics → canonical PlacePrep topic list. Original text preserved in `raw_text` field for audit and re-normalisation.

6 Step 6 — Entity Extraction — LLM Assisted

For unstructured text (Reddit posts, GFG experiences, NST Form submissions): send to LLM (claude-haiku-4-5 or gpt-4o-mini) with a structured extraction prompt. Extract JSON: {company_name, role_title, rounds_count, topics_asked[], questions_asked[], overall_difficulty, outcome, date_approx}. Parse LLM JSON response; merge extracted fields into the record. Token budget: 2,000 input + 400 output per record. Batch 10 records per API call with 1-second delay. Cache responses by `SHA256(input_text)` to avoid re-processing identical records.

7 Step 7 — Topic Classification — LLM

Every interview question is classified into one canonical topic from the PlacePrep taxonomy. Prompt: "Classify this interview question into exactly ONE of: Arrays, Linked Lists, Trees, Graphs, Dynamic Programming, Greedy, Sorting, Binary Search, Recursion, Bit Manipulation, System Design, Low Level Design, SQL/DBMS, OS Concepts, Networking, OOP, Behavioural, Aptitude-Logical, Aptitude-Quantitative, Aptitude-Verbal, Other." Use a fine-tuned local classifier (scikit-learn + sentence-transformers) for >95% confidence cases to reduce LLM API cost. Route only ambiguous cases to the LLM.

8 Step 8 — Difficulty Detection & Structured Output

Assign difficulty using multi-factor model: (1) If source provides difficulty (LeetCode CSV, Codeforces rating) — use directly. (2) Else: LLM confidence score + company tier (FAANG companies elevate difficulty one level) + community upvotes/frequency. Final: easy / medium / hard. After assignment, write to PostgreSQL (all fields), index in Elasticsearch (searchable fields only), embed via Embeddings API and store vector in pgvector (canonical records only). Log `structured_output_count` metric to Airflow XCom for monitoring.

LLM Cost & Usage Guidelines for ETL

- **Model:** claude-haiku-4-5 or gpt-4o-mini — cost-efficient for batch ETL. Reserve Claude Sonnet/GPT-4o for RAG user queries.
- **Estimated cost:** \$2–5 per 10,000 records for entity extraction + classification. Set a monthly ETL LLM budget alert at \$20.
- **Fallback:** If LLM call fails (timeout, rate limit), mark classification as "unclassified" and add to re-queue table for next ETL cycle.
- **Cache:** All LLM responses cached by `SHA256(input_text)` in a `lru_cache` table. Prevents re-billing for identical records across ETL runs.

7. Section 4: Storage Architecture

PlacePrep uses four specialised storage systems. No single database handles all requirements efficiently — this multi-database architecture is intentional. Each engine has a specific job that the others cannot do as well.

7.1 PostgreSQL — Primary Database

PostgreSQL is the master source of truth for all structured data. Two schemas: staging (raw ETL output; may have duplicates) and production (clean, deduped, served to API). ETL promotion job runs nightly.

Table	Key Fields	Purpose	Est. Rows
companies	id, name, type, tier, hq_location, is_nst_recruiter	Master company registry. All questions/experiences link here.	~2,000
roles	id, company_id, title, level (SDE-1/2/3), department	Role-specific question and experience mapping.	~8,000
interview_questions	id, question_text, raw_text, topic_id, difficulty, source_id, source_priority, is_duplicate, canonical_id, frequency_score	Core question bank. Most frequently queried table in the system.	~80,000
interview_rounds	id, company_id, role_id, round_type (enum), typical_duration_min	Round structure per company/role combination.	~12,000
topics	id, name, category (DSA/SD/OS/etc), parent_topic_id	Canonical topic taxonomy. Hierarchical (Arrays → Sliding Window).	~150
skills	id, name, category (language/framework/tool), demand_score	Skill tags for curriculum gap analysis (Faculty Dashboard).	~500
interview_experiences	id, user_id, company_id, role_id, experience_text, rounds_count, outcome, source_id, source_url, submitted_at	Real interview narratives from NST Form + GFG + Reddit.	~15,000
users	id, email (NST only), role (student/faculty/admin), batch_year, branch	NST students and faculty. Restricted to @nst.edu email domain.	~2,500
curriculum_topics	id, course_name, semester, topic_id, taught_depth (1–5)	NST curriculum mapping. Powers Faculty Dashboard gap analysis.	~800
company_frequencies	question_id, company_id, frequency_pct, last_updated	Frequency % per question per company. Drives "Most Asked" rankings.	~200,000
user_progress	user_id, question_id, status (solved/attempted/bookmarked), score, solved_at	Individual student progress tracking per question.	~500,000+

7.2 Elasticsearch — Search Engine

Elasticsearch handles all full-text search queries. A student typing "binary search in Amazon" gets results in milliseconds. Elasticsearch is a read-only replica — only the ETL pipeline writes to it; the API never writes directly.

Index Configuration

Index	Fields & Configuration
questions	question_text (text, English stemmer), topic (keyword), difficulty (keyword), companies[] (keyword), frequency_score (float). Custom synonym filter: "hashtable" = "hash map", "hashmap".
companies	name (text + keyword), type (keyword), tier (integer), interview_process_desc (text). Used by Company Explorer.
topics	name (text + keyword), category (keyword), question_count (integer). Used for topic browsing and autocomplete suggestions.
experiences	company_name (text), role (text), experience_text (text), outcome (keyword), source (keyword). Used for Experience Feed.

- **Refresh interval:** 30 seconds. Acceptable lag for interview prep content.
- **Reindex trigger:** Any ETL cycle producing >100 new canonical records triggers an incremental index update.

7.3 pgvector — AI / Vector Database

pgvector is a PostgreSQL extension that stores embedding vectors (lists of floats encoding semantic meaning). It powers the AI Assistant (RAG) and the "similar questions" feature. Only canonical (non-duplicate) records get embeddings.

pgvector Setup & Embedding Strategy

- **Install:** CREATE EXTENSION vector; — available in PostgreSQL 14+.
- **Model:** text-embedding-3-small (OpenAI) — 1536 dimensions. All embeddings must use the same model; mixing models breaks similarity scores.
- **Tables:** question_embeddings (question_id FK, embedding vector(1536)), experience_embeddings (experience_id FK, embedding vector(1536)), topic_embeddings (topic_id FK, embedding vector(1536)).
- **Index:**

```
CREATE INDEX ON question_embeddings
  USING ivfflat (embedding vector_cosine_ops)
  WITH (lists=100);
```

Query pattern:

```
SELECT q.id, q.question_text
FROM question_embeddings qe
JOIN interview_questions q ON q.id = qe.question_id
ORDER BY qe.embedding <=> :query_vector
LIMIT 10;
```

- Re-embedding required if model changes. Never mix embeddings from different models in the same table.
- Embedding generation is the final ETL step — runs only after dedup confirms canonical=TRUE.

7.4 Redis — Cache Layer

Redis is an in-memory store. It caches pre-computed results that are expensive to recalculate and frequently requested, ensuring fast API responses under concurrent load.

Cache Key Pattern	Value Stored	TTL	Invalidation Trigger
company:{id}:profile	Full company profile JSON with round structure and top 20 questions	24h	New company data ingested via ETL
topic:{id}:rankings	Top 20 questions by frequency for each topic	6h	ETL cycle completion
user:{id}:readiness	Pre-computed readiness score by company/role	1h	User solves or attempts a question
session:{token}	JWT payload, last_active timestamp, user role	30m (sliding)	User logout or token expiry
leaderboard:batch:{year}	Top 50 students by readiness score per batch year	2h	User completes a company roadmap module
question:freq:top50	Top 50 most-asked questions globally	12h	ETL cycle or manual cache flush

Redis Configuration

- **maxmemory:** 512mb — set `maxmemory-policy allkeys-lru` for automatic eviction of least-recently-used keys.
- **Persistence:** `appendonly yes` (AOF) — survives Redis restarts without full cache cold-start.
- **Security:** Never store sensitive PII in Redis. Session tokens contain only `user_id` and `role` — minimal exposure.
- **Cache warming:** After each ETL cycle, pre-warm company profile cache for the top 100 most-queried companies.

8. Implementation Checklist & Execution Order

Complete items in order. Each phase depends on the previous one being stable and tested. Do not skip phases even if you are under time pressure.

Phase 1 — Infrastructure Setup (Before Any Code)

- PostgreSQL: Create staging and production schemas. Enable `pg_trgm` and `pgvector` extensions.
- Elasticsearch/OpenSearch: Spin up local Docker instance. Create all four indexes (questions, companies, topics, experiences).
- Redis: Start local Docker instance. Configure maxmemory and eviction policy.
- Apache Airflow: Set up via Docker Compose (LocalExecutor for dev). Create `placeprep_data_pipeline` DAG skeleton.
- S3/MinIO: Create buckets — `placeprep-staging`, `placeprep-rejected`, `placeprep-archive`.
- Environment variables: `REDDIT_CLIENT_ID`, `REDDIT_CLIENT_SECRET`, `OPENAI_API_KEY`, `KAGGLE_JSON`, `DB_URL`, `ES_URL`, `REDIS_URL`.
- **MANDATORY:** All team members must read Section 2 (Legal Framework) before starting any collection work.

Phase 2 — Group D: Manual & Institutional Data (Week 1)

- Design and publish NST Google Form (Source 50) with all fields from §4.5.
- Distribute form link to NST students via placement group, alumni network, department WhatsApp.
- Send formal written request to NST Placement Cell (Source 51) for historical placement data.
- Upload first Google Form CSV export to `s3://placeprep-staging/50/`. Run ETL pipeline for the first time.
- Verify data appears in PostgreSQL staging schema with correct `source_id=50`.

Phase 3 — Group A: GitHub Repos & Datasets (Week 1–2)

- Clone all 26 GitHub repos (Sources 1–26) to `/data/staging/github/{source_id}/`.
- Set up Kaggle CLI; download Sources 27–30 CSV files to `/data/staging/kaggle/`.
- Run `load_dataset()` for HuggingFace Sources 31–32; export CSVs to `/data/staging/hf/`.
- Run full ETL pipeline for all Group A sources. Validate row counts match expectations (~60k+ questions).
- Verify deduplication is working: check `is_duplicate=TRUE` counts between Sources 1, 2, 3 (all LeetCode data).
- Confirm Elasticsearch has been populated and test 5 search queries manually.

Phase 4 — Group C: Official APIs (Week 2)

- Register Reddit developer app at `reddit.com/prefs/apps`. Store credentials in env vars.
- Run Reddit scraper for `r/cscareerquestions` (Source 45) — collect top 1,000 posts from the past year.
- Run Reddit scraper for `r/developersIndia` (Source 46) — collect top 500 India-specific posts.
- Call Codeforces API (Source 47) for all public problems. Map CF rating to PlacePrep difficulty.
- Run ETL pipeline for Group C API data. Verify entity extraction is correctly parsing Reddit posts.

Phase 5 — Group B: Static Scraping (Week 3)

- CHECK robots.txt for EACH source before writing its scraper. Document allow/disallow.
- Build and run IndiaBix scraper (Sources 35–37) — most reliable Group B source.
- Build and run CareerRide scraper (Source 42) — clean HTML, low risk.
- Build and run GFG experience scraper (Source 33) — experiences only, NOT raw question text.
- Build and run Preplnsta scrapers (Sources 38–41) — tag all records `caution_source=TRUE` in DB.
- Monitor rate limit errors throughout. Use exponential backoff if HTTP 429 received.

Phase 6 — Group C: Playwright (Week 3–4)

- Build AmbitionBox scraper (Source 43). Tag all records `manually_curated=FALSE`, `caution_source=TRUE`.
- Build Unstop competition listings scraper (Source 49). Public listings only.
- **DO NOT** build Naukri scraper (Source 48). Use Placement Cell data instead.
- **DO NOT** build InterviewBit login scraper (Source 44). Add manual seed records only.
- Run ETL for Group C Playwright data. Verify all caution-tagged records are excluded from public API responses until reviewed.

Phase 7 — QA & Handoff (Week 4)

- Verify total counts: ~80,000 interview questions, ~15,000 experiences, ~2,000 companies.
- Run full deduplication scan. Target: <15% duplicate rate across question bank.
- Manually inspect 50 random records from each source group. Spot-check topic classification and difficulty.
- Confirm all records have `source_id` populated. Zero orphan records allowed.
- Run Elasticsearch index refresh. Test 10 diverse search queries.
- Generate 100 test embeddings in pgvector. Run similarity query and verify results make semantic sense.
- Load Redis with top 100 company profiles. Verify cache hit rate on second request.
- Document any sources that produced significantly less data than expected. Flag for re-scrape.
- Hand this document + a data volume summary sheet to the backend/API team.

References & Key Links

External Documentation

Resource	Link	Install / Note
Reddit API	reddit.com/prefs/apps	PRAW docs: praw.readthedocs.io
Kaggle API	kaggle.com/docs/api	<code>pip install kaggle</code>
HuggingFace	huggingface.co/docs/datasets	<code>pip install datasets</code>
Playwright	playwright.dev/python	<code>pip install playwright</code> && <code>playwright install chromium</code>
pg_trgm	postgresql.org/docs/current/pgtrgm.html	—
pgvector	github.com/pgvector/pgvector	—
Apache Airflow	airflow.apache.org/docs/	—

Resource	Link	Install / Note
NST Placement Cell	Contact: [To be filled by project lead before handoff]	—

PlacePrep — NST Interview Intelligence Portal | Data Pipeline Technical Documentation v1.0 | June 2026 | Newton School of Technology, Rishihood University. This document is confidential and intended for internal development use only. Review required before any commercial launch of the platform.